1 Controling the Robot via Potentiometer

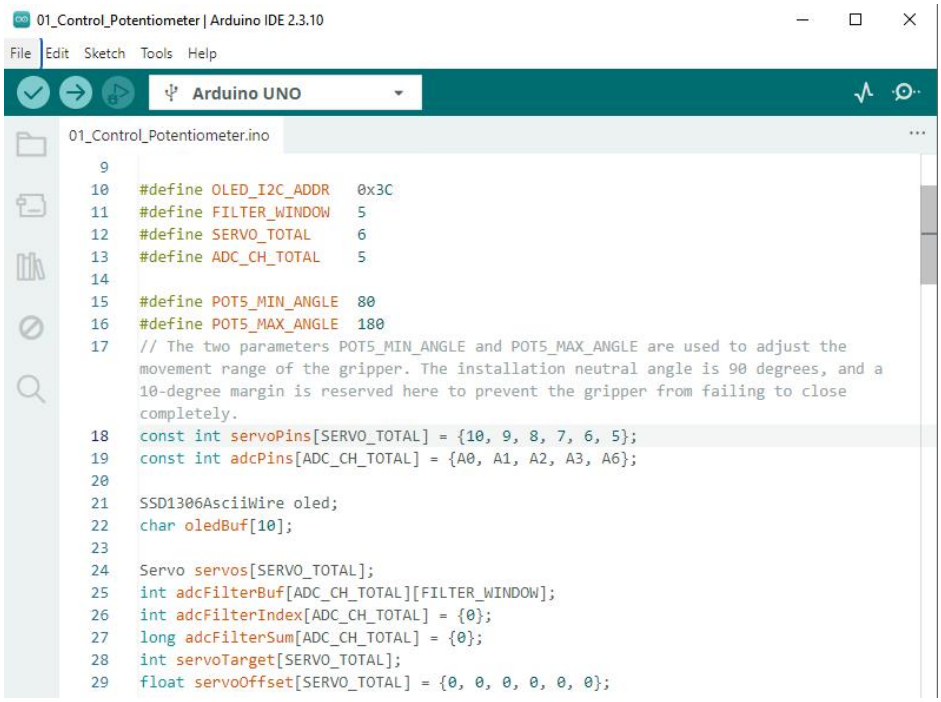
1.1 Overview

You have fully assembled the robot arm. In this lesson, we will learn to control it with potentiometers. The five potentiometers on the control board control the five servos S6 to S2 in sequence, enabling the robot arm to move freely and flexibly.

1.2 Upload infrared control program

1. Connect your computer and Adeept Robotic Arm Control Board with a USB cable. Then connect the battery and turn on the power switch.

2. Open "**01_Control_Potentiometer**" folder in "**/Code**", double-click "**01_Control_Potentiometer.ino**".



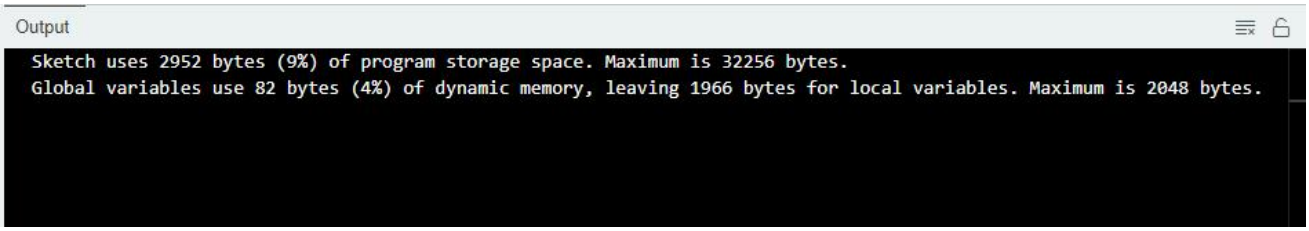
3. Select development board and serial port.

Board: **Tools--->Board--->Arduino AVR Boards--->Arduino Uno**

Port: **Tools --->Port--->COMx**

Note: The port number will be different in different computers.

4. After opening, click  to upload the code program to the Arduino. If there is no error warning in the console below, it means that the Upload is successful.



5. You will now see the angles of the corresponding servos for each potentiometer displayed on the OLED screen. Turn the potentiometers to control the movement of the robotic arm, and the angle values on the OLED will change in real time.

1.3 Code

```
/*************************************************************
Description: Potentiometer controls servo angle with filter & stable OLED
Website: www.adeept.com
E-mail: support@adeept.com
*************************************************************/
#include <Wire.h>
#include <SSD1306AsciiWire.h>
#include <Servo.h>

#define OLED_I2C_ADDR   0x3C
#define FILTER_WINDOW   5
#define SERVO_TOTAL     6
#define ADC_CH_TOTAL    10
#define PIN_BATTERY     A7
#define PIN_BUZZER 3
#define BUZZER_FREQUENCY 4000

#define POT5_MIN_ANGLE  80
#define POT5_MAX_ANGLE  180
// The two parameters POT5_MIN_ANGLE and POT5_MAX_ANGLE are used to adjust the movement range of the
gripper. The installation neutral angle is 90 degrees, and a 10-degree margin is reserved here to
prevent the gripper from failing to close completely.
const int servoPins[SERVO_TOTAL] = {10, 9, 8, 7, 6, 5};
const int adcPins[ADC_CH_TOTAL] = {A0, A1, A2, A3, A6};

const float ADC_REF_VOLTAGE    = 5.1;
const float VOLTAGE_DIV_RATIO  = 2.0;
const float BAT_MIN_VOLTAGE    = 6.0;
```

```
029 const float BAT_MAX_VOLTAGE    = 8.4;
030 float batteryVoltage = 0.0;
031 unsigned long oledTime;
032
033 SSD1306AsciiWire oled;
034
035 Servo servos[SERVO_TOTAL];
036 int adcFilterBuf[ADC_CH_TOTAL][FILTER_WINDOW];
037 int adcFilterIndex[ADC_CH_TOTAL] = {0};
038 long adcFilterSum[ADC_CH_TOTAL] = {0};
039 int servoTarget[SERVO_TOTAL];
040 int lastOledVal[ADC_CH_TOTAL] = {-1, -1, -1, -1, -1};
041
042 void adcFilterInit() {
043   for (int ch = 0; ch < ADC_CH_TOTAL; ch++) {
044     adcFilterSum[ch] = 512L * FILTER_WINDOW;
045     for (int i = 0; i < FILTER_WINDOW; i++) {
046       adcFilterBuf[ch][i] = 512;
047     }
048   }
049 }
050
051 int adcFilter(int channel, int rawVal) {
052   adcFilterSum[channel] -= adcFilterBuf[channel][adcFilterIndex[channel]];
053   adcFilterBuf[channel][adcFilterIndex[channel]] = rawVal;
054   adcFilterSum[channel] += rawVal;
055   adcFilterIndex[channel] = (adcFilterIndex[channel] + 1) % FILTER_WINDOW;
056   return (int)(adcFilterSum[channel] / FILTER_WINDOW);
057 }
058
059 void oledInit() {
060   Wire.begin();
061   oled.begin(&Adafruit128x64, OLED_I2C_ADDR);
062   oled.setFont(Adafruit5x7);
063   oled.clear();
064
065   oledPrintStr(5,0,"Battery Percent:");
066   oledPrintStr(0, 2, "A0:");
067   oledPrintStr(65, 2, "A1:");
068   oledPrintStr(0, 4, "A2:");
069   oledPrintStr(65, 4, "A3:");
070   oledPrintStr(0, 6, "A6:");
071 }
072
073 void oledPrintNum(int x,int y,int num){
074   oled.set1X();
075   oled.setCursor(x,y);
076   oled.print(F("     "));
077   oled.setCursor(x,y);
078   oled.print(num);
079 }
080
081 void oledPrintStr(int x,int y,const char *text){
082   oled.set1X();
083   oled.setCursor(x,y);
084   oled.print(F("                    "));
```

```
085   oled.setCursor(x,y);
086   oled.print(text);
087 }
088
089 void servoInit() {
090   for (int i = 0; i < SERVO_TOTAL; i++) {
091     servos[i].attach(servoPins[i]);
092   }
093 }
094
095 void setup() {
096   adcFilterInit();
097   servoInit();
098   oledInit();
099   Buzzer_Setup();
100   Buzzer_Alert(2,1);
101 }
102
103 void loop() {
104   int filteredAdc[ADC_CH_TOTAL];
105   for (int ch = 0; ch < ADC_CH_TOTAL; ch++) {
106     int raw = analogRead(adcPins[ch]);
107     filteredAdc[ch] = adcFilter(ch, raw);
108   }
109
110   for (int i = 0; i < 4; i++) {
111     servoTarget[i] = map(filteredAdc[i], 0, 1023, 0, 180);
112   }
113   // Potentiometer A6: It limits the angle control range of the gripper and synchronously controls
114 Servos 5 and 6 to prevent the servos from being damaged due to stall.
115   int angle5 = map(filteredAdc[4], 0, 1023, POT5_MIN_ANGLE, POT5_MAX_ANGLE);
116   servoTarget[4] = angle5;
117   servoTarget[5] = angle5;
118
119   for (int i = 0; i < SERVO_TOTAL; i++) {
120     servos[i].write(servoTarget[i]);
121     delay(5);
122   }
123
124   OLED_display();
125 }
126
127 void OLED_display(){
128   if(millis()-oledTime<1000) return;
129   oledTime=millis();
130   if (servoTarget[0] != lastOledVal[0]) {
131     oledPrintNum(20, 2, servoTarget[0]);
132     lastOledVal[0] = servoTarget[0];
133   }
134   if (servoTarget[1] != lastOledVal[1]) {
135     oledPrintNum(85, 2, servoTarget[1]);
136     lastOledVal[1] = servoTarget[1];
137   }
138   if (servoTarget[2] != lastOledVal[2]) {
139     oledPrintNum(20, 4, servoTarget[2]);
140     lastOledVal[2] = servoTarget[2];
```

```
  }
  if (servoTarget[3] != lastOledVal[3]) {
    oledPrintNum(85, 4, servoTarget[3]);
    lastOledVal[3] = servoTarget[3];
  }
  if (servoTarget[4] != lastOledVal[4]) {
    oledPrintNum(20, 6, servoTarget[4]);
    lastOledVal[4] = servoTarget[4];
  }

  int batRatio = Get_Battery_Ratio();
  oledPrintNum(100, 0, batRatio);
  oledPrintStr(115, 0, "%");
}


//////////////////////Buzzer drive area////////////////////////////////
void Buzzer_Setup(){
  pinMode(PIN_BUZZER, OUTPUT);
  digitalWrite(PIN_BUZZER, LOW);
}

void softTone(int durationMs){
  unsigned long start = millis();
  int delayUs = 500000 / BUZZER_FREQUENCY;
  while (millis() - start < durationMs) {
    digitalWrite(PIN_BUZZER, HIGH);
    delayMicroseconds(delayUs);
    digitalWrite(PIN_BUZZER, LOW);
    delayMicroseconds(delayUs);
  }
}

void Buzzer_Alert(int beat, int rebeat){
  beat = constrain(beat, 1, 9);
  rebeat = constrain(rebeat, 1, 255);
  for (int j = 0; j < rebeat; j++) {
    for (int i = 0; i < beat; i++) {
      softTone(100);
      delay(100);
    }
    delay(500);
  }
}


int Get_Battery_Voltage_ADC(){
  int adcValue = 0;
  for (int i = 0; i < 5; i++){
    adcValue += analogRead(PIN_BATTERY);
    delayMicroseconds(200);
  }
  return adcValue / 5;
}

float Get_Battery_Voltage(){
```

```
197   int adcValue = Get_Battery_Voltage_ADC();
198   batteryVoltage = ((float)adcValue / 1023.0) * ADC_REF_VOLTAGE * VOLTAGE_DIV_RATIO;
199   batteryVoltage = constrain(batteryVoltage, 0, BAT_MAX_VOLTAGE);
200   return batteryVoltage;
201 }
202
203 int Get_Battery_Ratio(){
204   float voltage = Get_Battery_Voltage();
      int ratio = map(voltage * 100, BAT_MIN_VOLTAGE * 100, BAT_MAX_VOLTAGE * 100, 0, 100);
      return constrain(ratio, 0, 100);
    }
```